Ideas FloodWatch App Using an already build AI model:

Software System for Satellite Data Analysis Using Artificial Intelligence for Flood Detection and Prediction

1. The Conversion Formula

To convert DMS to Decimal Degrees, you use this formula:

$$DD = \text{Degrees} + \left(\frac{\text{Minutes}}{60}\right) + \left(\frac{\text{Seconds}}{3600}\right)$$

(Note: North and East are positive numbers. South and West are negative numbers).

Revisit Interval (often called Temporal Resolution) is the time it takes for a satellite to complete its orbit and fly over the exact same spot on Earth again.

Cadence is the frequency at which a new "file" or "image" is generated and added to the dataset collection. Datasets with a cadence are often combinations of multiple satellites, weather stations, or mathematical models.

1. Prediction

    a. DEM (Digital Elevation Model) maps:

Name	Link	Pixel Size	Dataset Availability
Copernicus DEM GLO-30: Global 30m Digital Elevation Model	link	30 meters	**2010**-12-01T00:00:00Z–**2015**-01-31T00:00:00Z

    b. Precipitation real-time maps:

Name	Link	Pixel	Dataset Availability	Other

		Size		
CHIRPS Precipitation Daily Near-Real-Time	link	5566 meters	**1981**-01-01T00:00:00Z–**2026**-01-31T00:00:00Z	

c. SAR soil moisture:

Name	Link	Pixel Size	Dataset Availability	Other
Sentinel-1 SAR GRD	link	5566 meters	**2014**-10-03T00:00:00Z–**2026**-03-17T11:10:51Z	Revisit Interval - **6 Days**

2. Detection

Prebuilt AI model:

Name	Link	Data trained on	Model used
Prithvi-EO-1.0-100M-sen1floods11	link		
ai4g-flood	link	Sentinel-1 RTC SAR data from the Microsoft Planetary Computer, data includes radiometric calibration, terrain correction, and other custom data processing by Catalyst	**The flood detection model is based on a U-Net architecture with a MobileNetV2 encoder**

My application/ thesis will consist of two very important features: prediction and detection.

For prediction I will use precipitation maps combined with information from DEM maps and wetland areas from Sentinel1 SAR maps(by using backscatter). Using a threshold based system I will compute every pixel and it's probability of flooding and I will display areas with potential of flooding with yellow/ orange/ red as a measure for the intensity.

For detecting I will use Microsoft's *ai4g-flood* model, which is a SAR based model. Here is a paper on their model: https://www.nature.com/articles/s41467-025-60973-1

<u>More on how I aim to integrate the ai4g-flood in my application:</u>

Important: This model was trained on Sentinel-1 RTC data from the Microsoft Planetary Computer sourced from Catalyst

RTC = Radiometric Terrain Correction

The SAR images need preprocessing before feeding them to the ai4g-flood model and the resulting images need postprocessing to mask the already existing water bodies that will get flagged with 1 by the model.

Preprocessing:

- Images taken from Copernicus need terrain correction, speckle filtering, and radiometric calibration (Copernicus Sentinel 1 SAR data already)

** do a full data flow diagram
** do a data processing diagram for the ai4g-flood model

<u>Why I choose SAR data:</u>

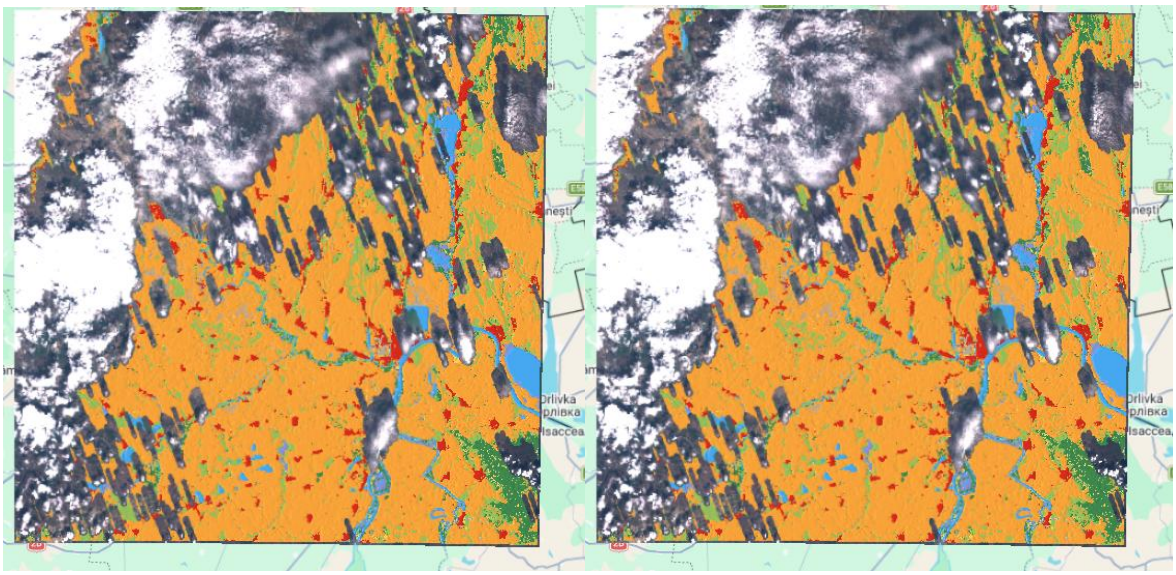
I have found SAR data to be more useful because it isn't affected by cloud cover (which is a very important factor when working with heavy rain).

<u>Why I didn't choose other datasets as my main data sources:</u>

DynamicWorld Dataset:

- During the rainy periods clouds cover the images and makes viewing of the flood affected regions impossible

Image example taken during a rainy week:



<u>Downloading Galati specific flood data corresponding to an actual flood from september 2024 during the Cyclone Boris.</u>

```
# 1. Define Galați Bounding Box
galati_roi = ee.Geometry.Rectangle([27.20, 45.25, 28.30, 46.25])
```

```
# Pre-flood: First week of September (Dry)
pre_flood_img = get_s1_mosaic('2024-09-01', '2024-09-10')

# Post-flood: Mid-September (During Cyclone Boris)
post_flood_img = get_s1_mosaic('2024-09-14', '2024-09-20')
```

```
def export_to_drive(image, filename):
    print(f"Starting export for {filename}...")
    task = ee.batch.Export.image.toDrive(
        image=image,
        description=filename,
        folder='Thesis_S1_Data', # A folder that will be created in your Google Drive
        region=galati_roi,
        scale=10, # 10-meter resolution (Sentinel-1 standard)
        maxPixels=1e10
    )
    task.start()
```

Conclusions:

- Saving the data in my personal google account takes a while (52 min).

  SUBMITTED TASKS

Galati_Post_VH	✓ 18m
Galati_Post_VV	✓ 11m
Galati_Pre_VH	✓ 13m
Galati_Pre_VV	✓ 10m

  Every file waits for the previous to compile and only then starts the download.

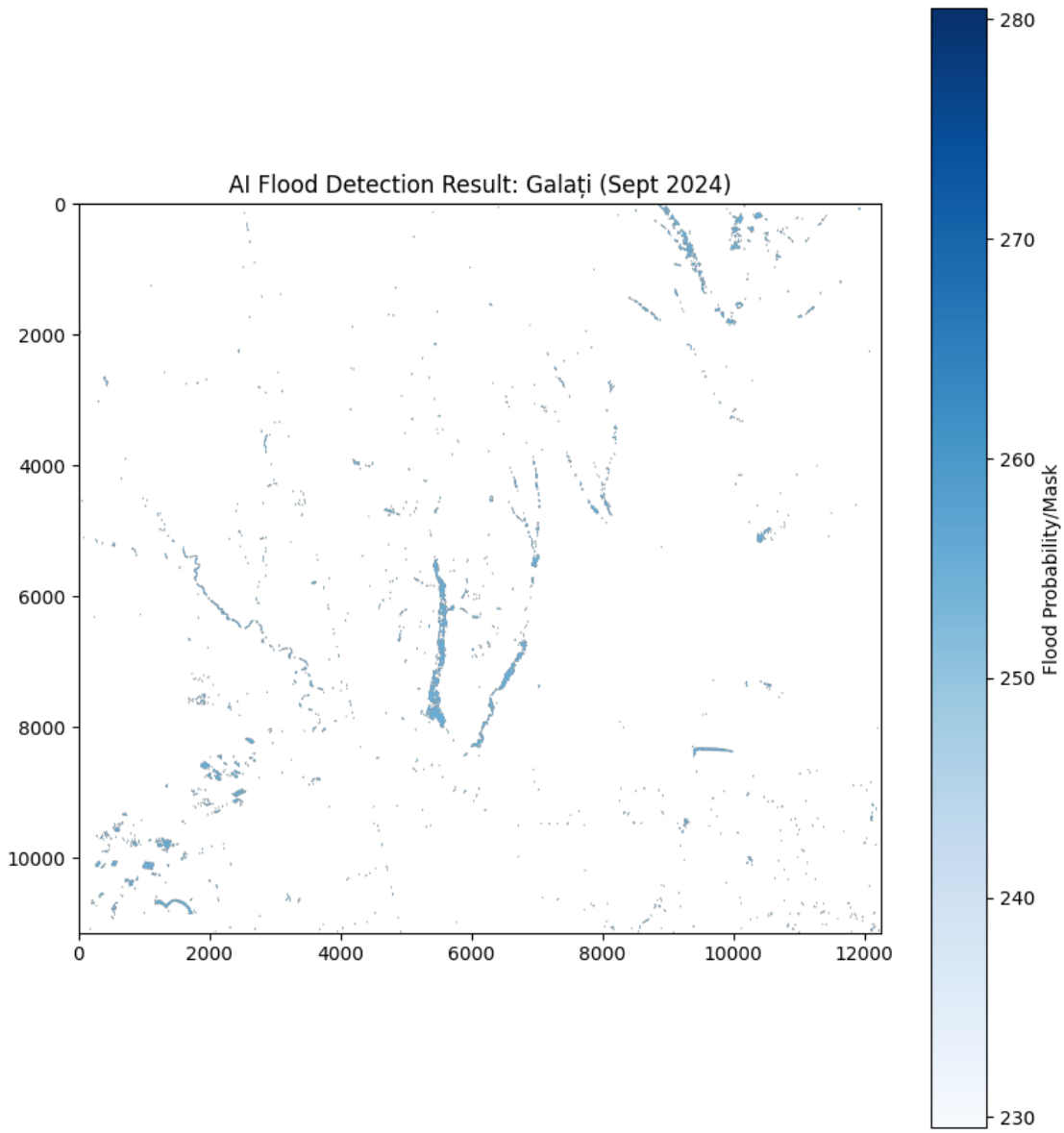
- If you want to download to your local computer you need to scale down the images (to about 50m or grater). However this makes the Galati region full of NaN values **->** I get the NaN values because the satellite images from Earth Engine are in db while it should be in linear values (the same values on which the ai model has been trained on).

```
============================================================
🏁 [FINISH] Pipeline completed at: 2026-04-09 01:46:17
⏱️  [DURATION] Total Execution Time: 0h 8m 27.24s
============================================================
```

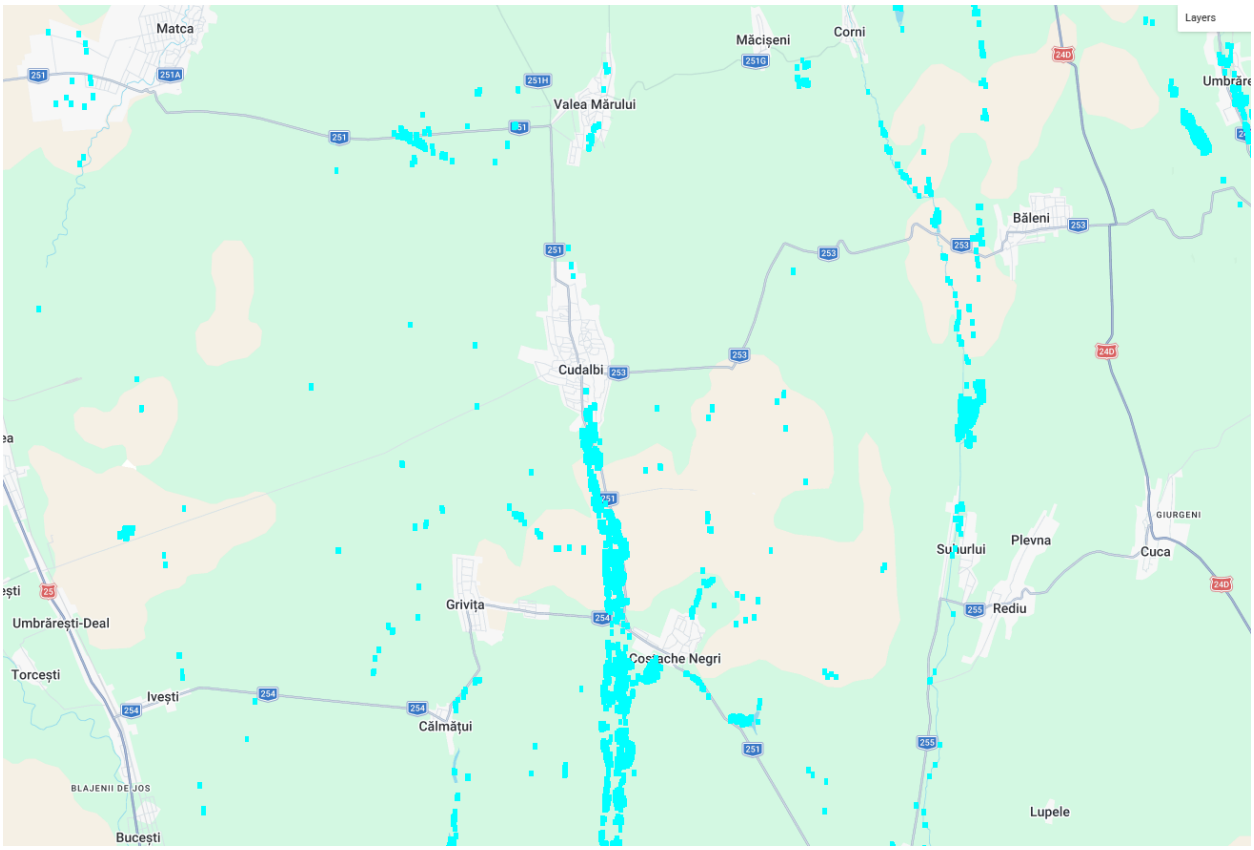
Running the model on the linear data took me 8 to 9 minutes.

This is the result when visualised:

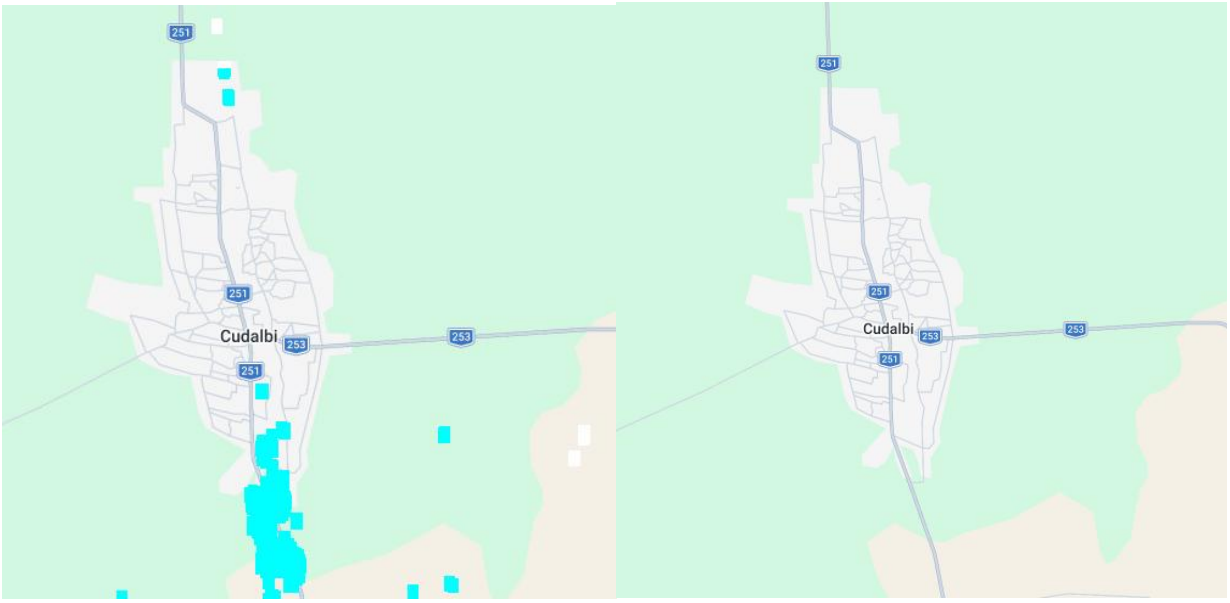
```
C:\Users\chior\Desktop\FACULTATE\licenta\ai4g-flood>py display_result.py
Unique values in output: [255.  nan]
Mean value: nan
Max value: nan
```



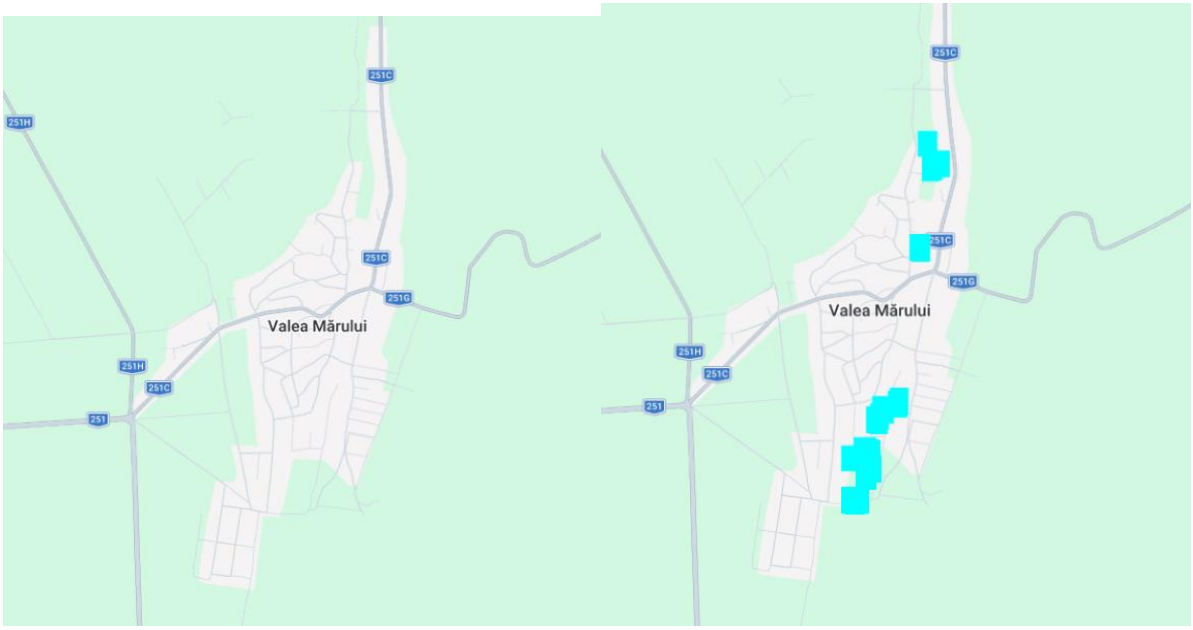
On a closer inspection we can see the affected regions in Galati:



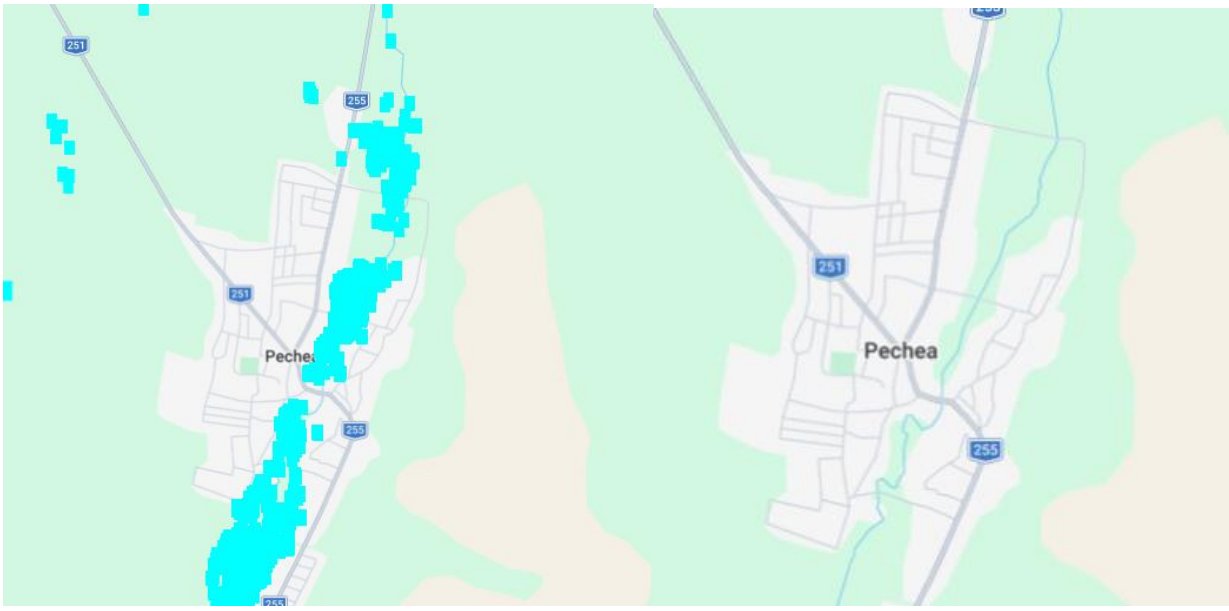
It is obvious that the Cudalbi village has been flooded:



Valea Marului as well:



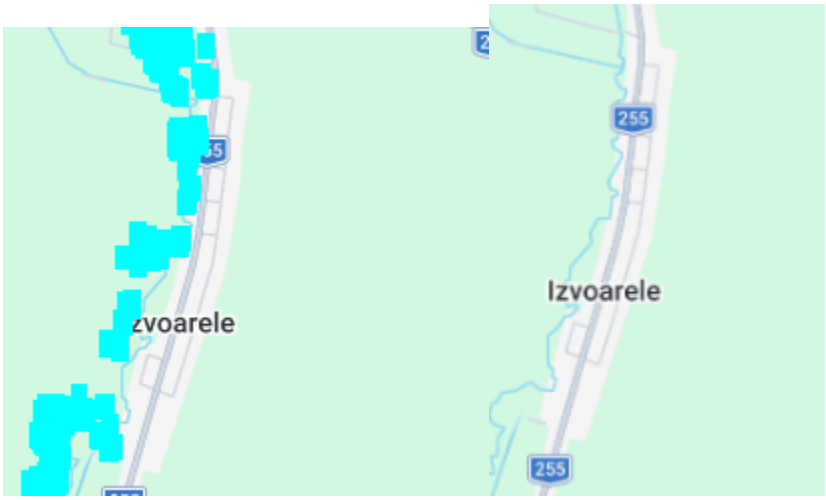
Pechea village:

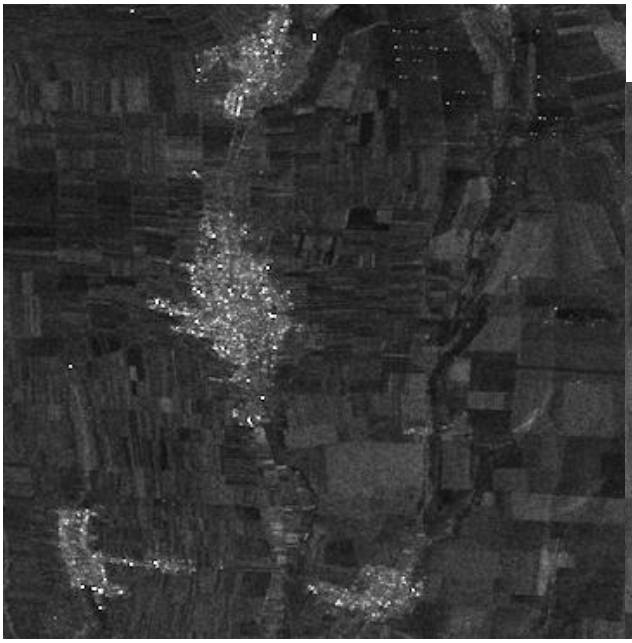


Cuza Voda village:

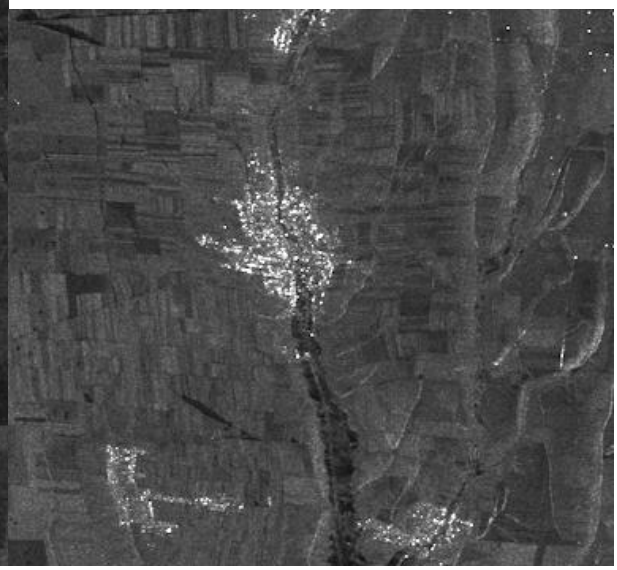
Slobozia Conachi village:

Izvoarele village:

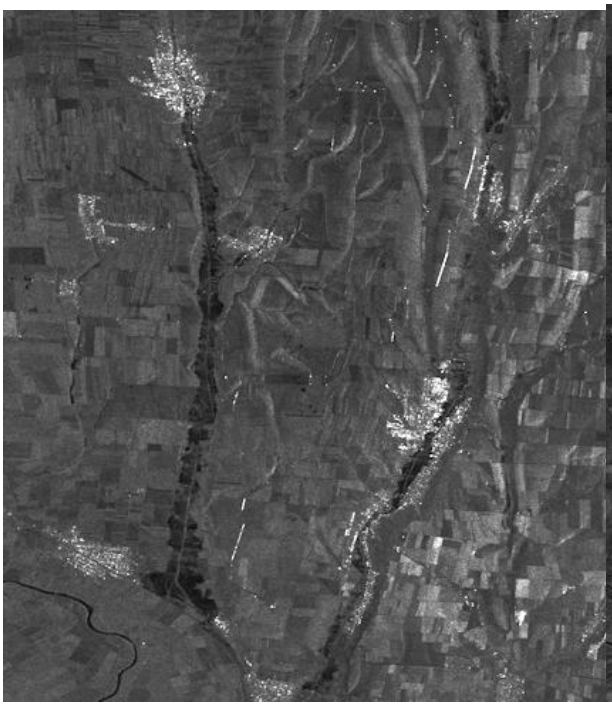




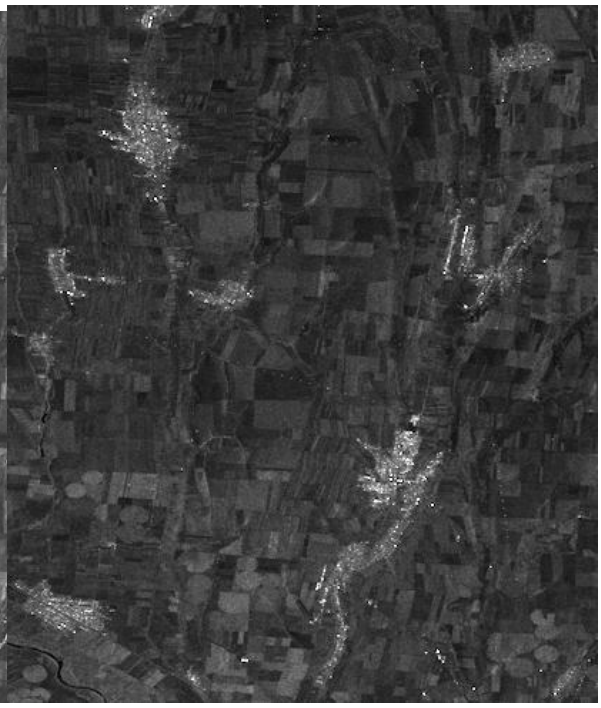
Pre flooding event



Post flooding event

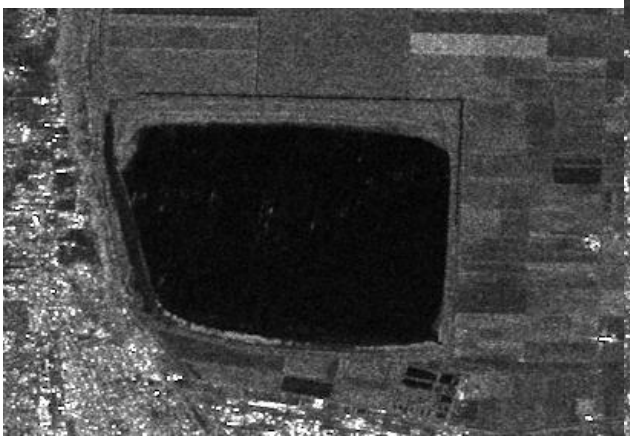


After flood

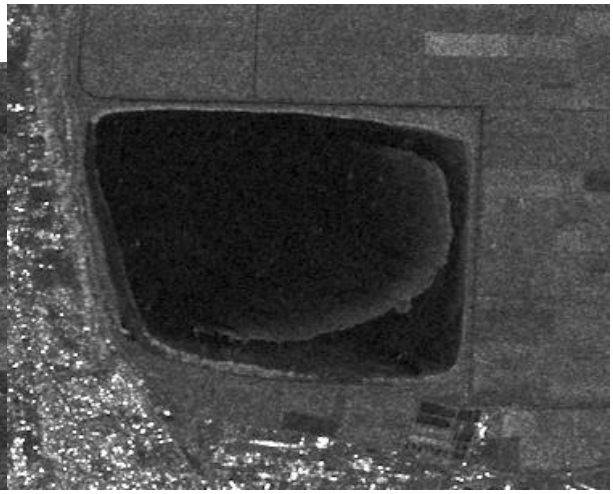


before the flood

Before



after



detected

Overview

Flooding is a natural disaster that is hard to predict and causes major economic and environment looses.

In Romania recent years, floods have affected communities. The application centers around cyclone Boris impact in Galati county Romania.

In 2019, the Ministry of the Environment, Waters and Forests (MMAP), together with the National Administration "Romanian Waters" (ANAR) and the National Institute of Hydrology and Water Management (INHGA) defined “Areas of Potential Significat Flood Risk”, documenting areas of flood risk in Romania.

Current Romanian systems that are available to the public rely on static maps, and they do not provide automated real-time analysis, which can hinder the reaction time of citizens and authorities.

I aim to implement an application that will offer smart solutions for detecting early signs of floods and to be able to predict a possible inundation in Romania.
It will monitor the regions from the the 526 Areas of Potential Significant Risk (APSFR) that are scattered around the river courses within Romanian territory. Because of hardware constrains, this thesis will only focus on the flooding event that happened in september 2024 in Galati county due to cyclone Boris.

The application name is ‘FloodWatch’ and will have two very important features: prediction and detection.

The prediction aspect of the application aims to catch early signs of flooding. It will compute the probability of flooding by using information from different satellite data sources with a probability function. When there is a cluster of possible inundation pixel results, the application will display on the map the region with a different colour, indicating the possibility of flood and severity of the impact. The areas will be color coded as follows: green-0, yellow-1, orange-2, red-3. For the prediction function it will take parameters data from three different satellite maps. It will use real time precipitation maps, DEM elevation maps and will use the backscatter effect from Sentinel 1 SAR images to determine wetlands.
The revisit interval for each map: for the precipitation map the algorithm will look at it every 10 minutes, for the DEM map it will refresh once every 6 months and Sentinel 1 SAR data will be refreshed once every 20 minutes.

When the detection algorithm shows high number and alerts are popping, then the detection algorithm will be started.
The detection algorithm uses Microsoft’s *ai4g-flood* model, which is a SAR based ai model. This will output a map of already flooded areas.